

Modelado de Sistemas Autónomos: Del Concepto al Código

Un análisis paso a paso de $\dot{x} = x^2 - 1$ con Python

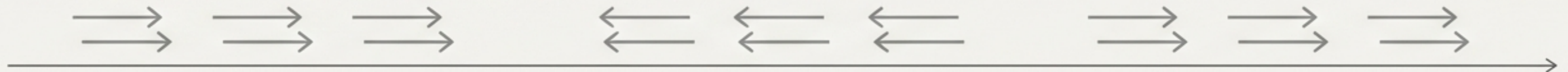
¿Hacia Dónde Evoluciona el Sistema?

Los sistemas autónomos describen la evolución de una variable x a lo largo del tiempo t mediante una ecuación $dx/dt = f(x)$, donde la tasa de cambio solo depende del estado actual.

$$\frac{dx}{dt} = x^2 - 1$$

Nuestro caso de estudio es el sistema:

****La pregunta clave****: Sin resolver la ecuación diferencial para cada posible condición inicial, ¿podemos predecir el comportamiento del sistema a largo plazo?
¿Existen estados de equilibrio?



Concepto Clave 1: El Equilibrio del Sistema

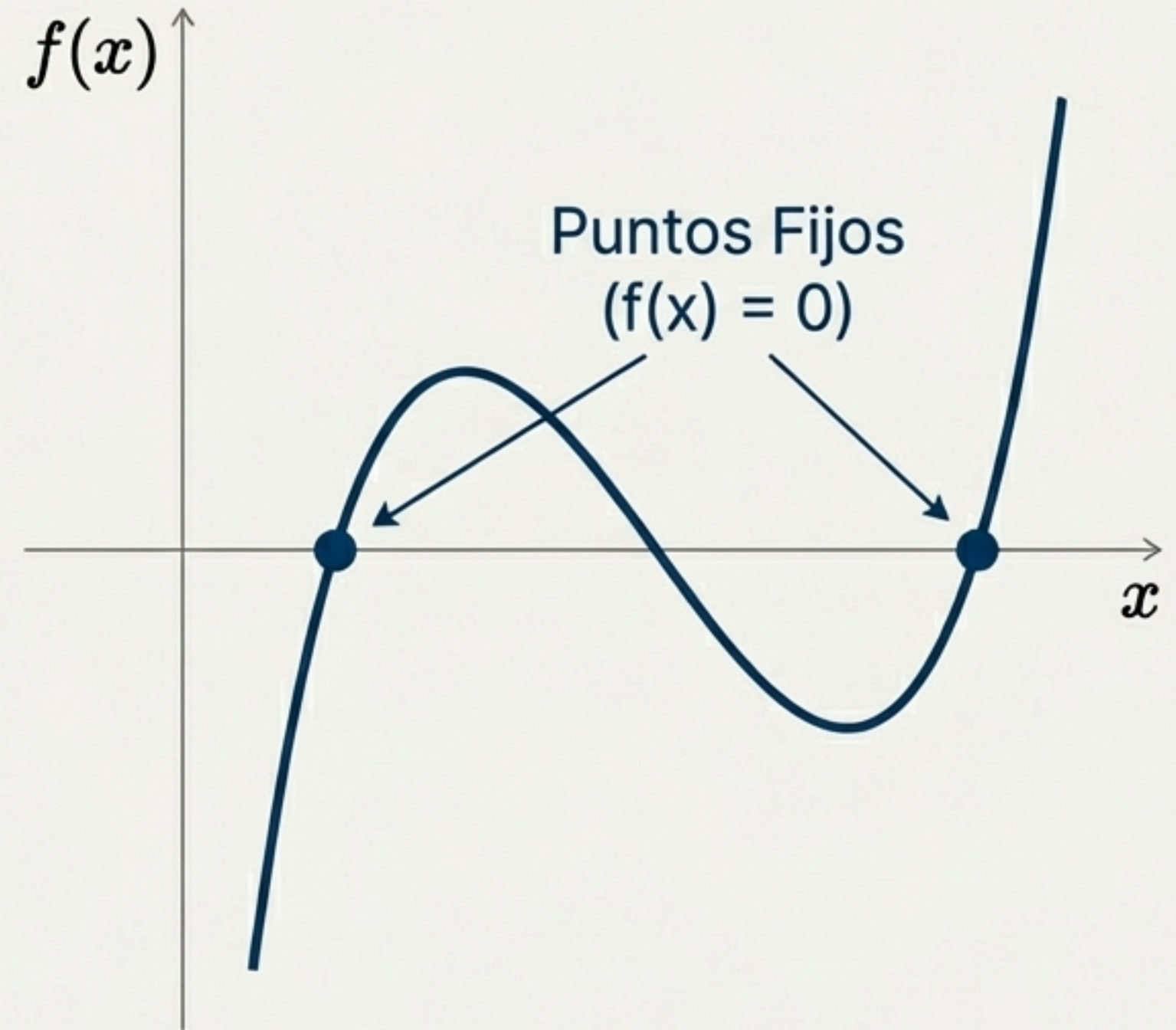
La base de nuestro análisis son los **puntos fijos** (o puntos de equilibrio).

Estos son los valores de x donde el sistema deja de evolucionar, es decir, donde la tasa de cambio es cero.

Matemáticamente, los encontramos resolviendo la ecuación: $f(x) = 0$.

Para nuestro sistema: $x^2 - 1 = 0$.

Estos puntos anclan la dinámica completa del sistema.



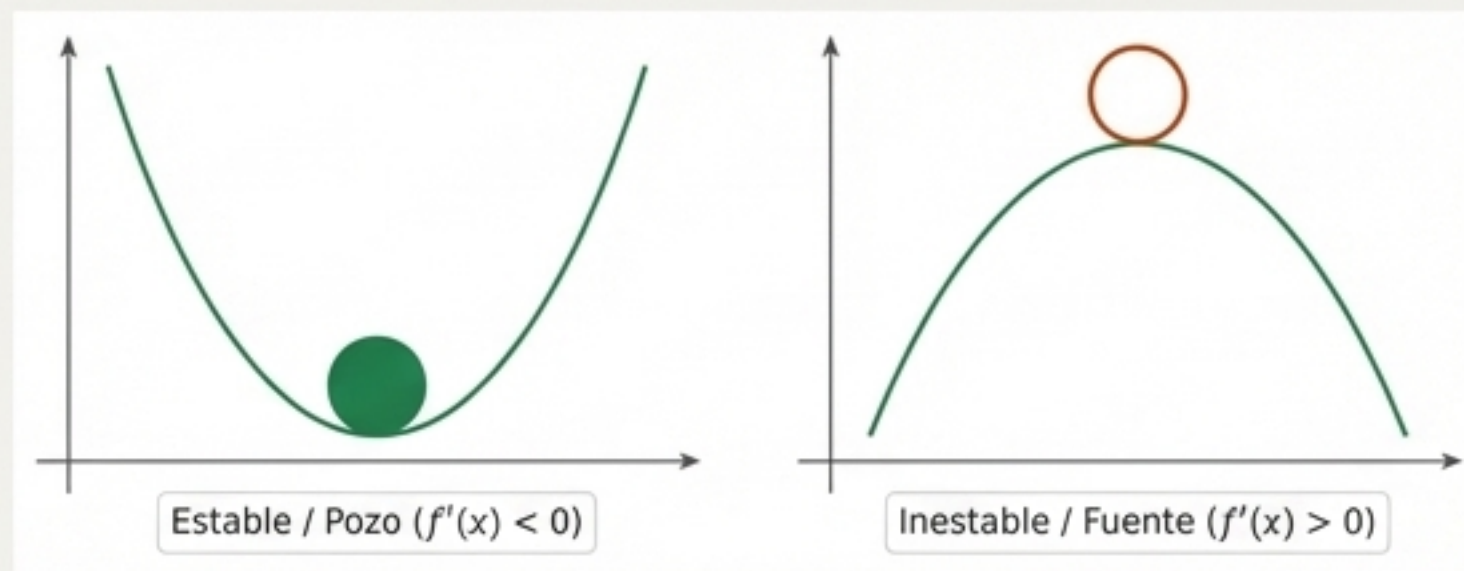
Concepto Clave 2: La Estabilidad en la Vecindad

No todos los puntos fijos son iguales. Su estabilidad determina el comportamiento del sistema cerca de ellos.

- **Estable** (Pozo): Los estados cercanos son atraídos hacia el punto fijo. Análogo a una bola en el fondo de un valle.
- **Inestable** (Fuente): Los estados cercanos son repelidos lejos del punto fijo. Análogo a una bola en la cima de una colina.

La prueba de la derivada: Analizamos el signo de la derivada, $f'(x)$, en el punto fijo.

- Si $f'(x) < 0$, el punto es **estable**.
- Si $f'(x) > 0$, el punto es **inestable**.



Nuestro Kit de Herramientas en Python

Para implementar nuestro análisis, utilizaremos tres bibliotecas fundamentales del ecosistema científico de Python:



NumPy

Para cálculo numérico eficiente y manejo de arrays. Es la base de nuestras operaciones.



SciPy

Proporciona herramientas científicas avanzadas. Usaremos sus módulos **optimize** (para encontrar raíces) e **integrate** (para resolver ecuaciones diferenciales).



Matplotlib

Para la creación de visualizaciones y gráficos de alta calidad, que nos permitirán "ver" el comportamiento del sistema.

Mapeando Conceptos a Funciones de Python

A continuación se muestra cómo cada paso de nuestro análisis teórico corresponde a una función específica en nuestro script:

****Concepto Teórico****: Definir el sistema
($\dot{x} = f(x)$)



****Implementación en Python****

```
def f(x):  
    return x**2 - 1
```

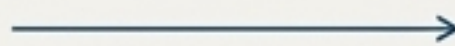
****Concepto Teórico****: Encontrar
Puntos Fijos ($f(x) = 0$)



****Implementación en Python****

```
scipy.optimize.fsolve(f, initial_guesses)
```

****Concepto Teórico****: Analizar
Estabilidad (evaluar $f'(x)$)



****Implementación en Python****

```
def f_prime(x):  
    return 2*x
```

****Concepto Teórico****: Simular la
evolución en el tiempo



****Implementación en Python****

```
scipy.integrate.solve_ivp(system, t_span, [x0])
```

Paso 1: Encontrando los Puntos Fijos

Usamos la función `fsolve` de SciPy para encontrar las raíces de $f(x) = x^2 - 1$. El código busca soluciones y filtra los resultados únicos para evitar duplicados.

Código

```
# Snippet de la función find_fixed_points
def find_fixed_points(f):
    # Búsqueda inicial de raíces
    fixed_points = fsolve(f, np.linspace(-2, 2, 5))
    # Filtro para obtener raíces únicas y precisas
    fixed_points = np.unique(fixed_points)
    true_fixed_points = []
    tolerance = 1e-6
    for point in fixed_points:
        if np.abs(f(point)) < tolerance:
            true_fixed_points.append(point)
    return np.array(true_fixed_points)
```

Salida del Programa:

```
Puntos Fijos:
x = -1.00
x =  1.00
```

Paso 2: Analizando la Estabilidad

Para cada punto fijo, calculamos la derivada $f'(x) = 2x$ y evaluamos su signo para determinar si el punto es un "Pozo" (estable) o una "Fuente" (inestable).

Código

```
# Snippet de la función stability_analysis
def stability_analysis(f, fixed_points):
    def f_prime(x):
        return 2 * x # Derivada de x^2 - 1
    stabilities = {}
    for point in fixed_points:
        derivative = f_prime(point)
        if derivative > 0:
            stabilities[point] = 'Inestable (Fuente)'
        elif derivative < 0:
            stabilities[point] = 'Estable (Pozo)'
    return stabilities
```

Salida del Programa:

```
x = -1.00, Naturaleza: Estable (Pozo)
x = 1.00, Naturaleza: Inestable (Fuente)
```

Interpretando Nuestros Resultados Analíticos

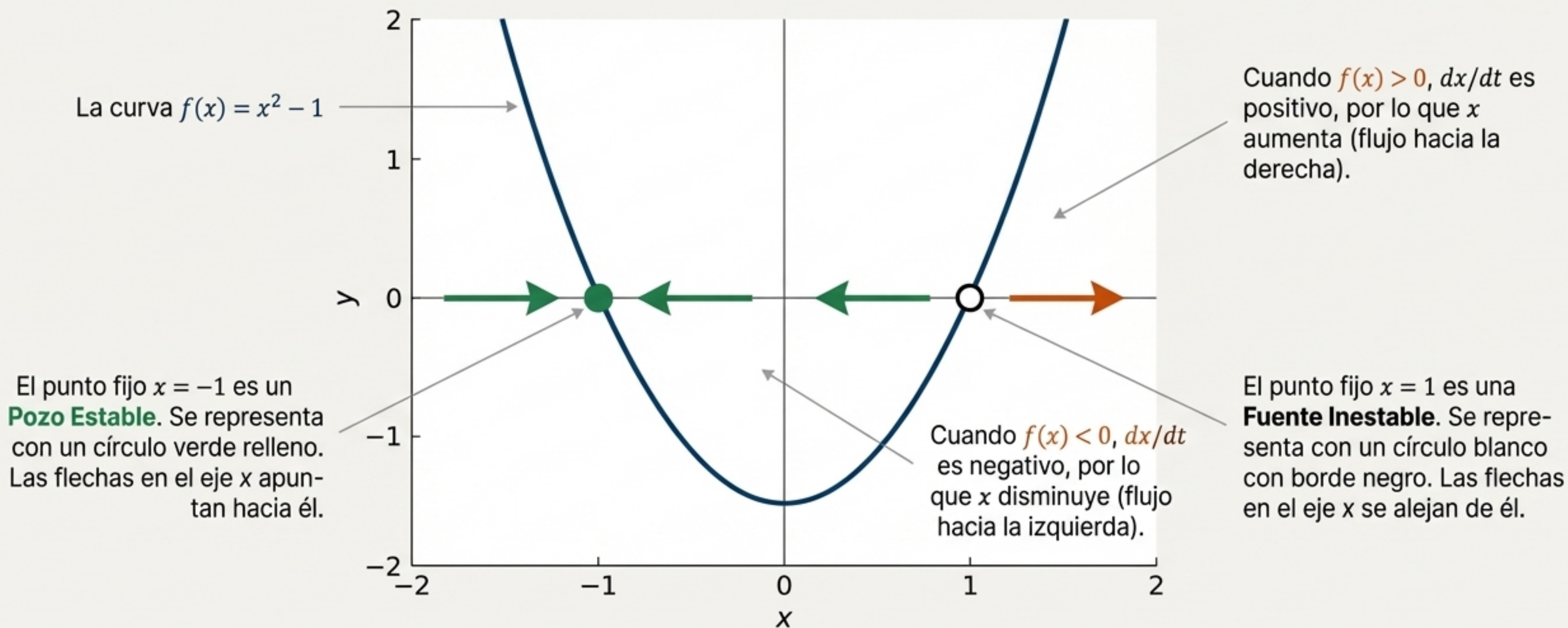
Hasta ahora, nuestro análisis nos dice todo lo que necesitamos para predecir la dinámica:

- Hemos identificado **dos puntos de equilibrio**: $x = -1$ y $x = 1$.
- El punto $x = -1$ es **estable (Pozo)**. Esperamos que las trayectorias que comiencen cerca de -1 converjan hacia él.
- El punto $x = 1$ es **inestable (Fuente)**. Esperamos que las trayectorias que comiencen cerca de 1 se alejen de él.

Siguiente paso: Visualizar esta dinámica para confirmar nuestras predicciones.

Visualización 1: El Retrato de Fase

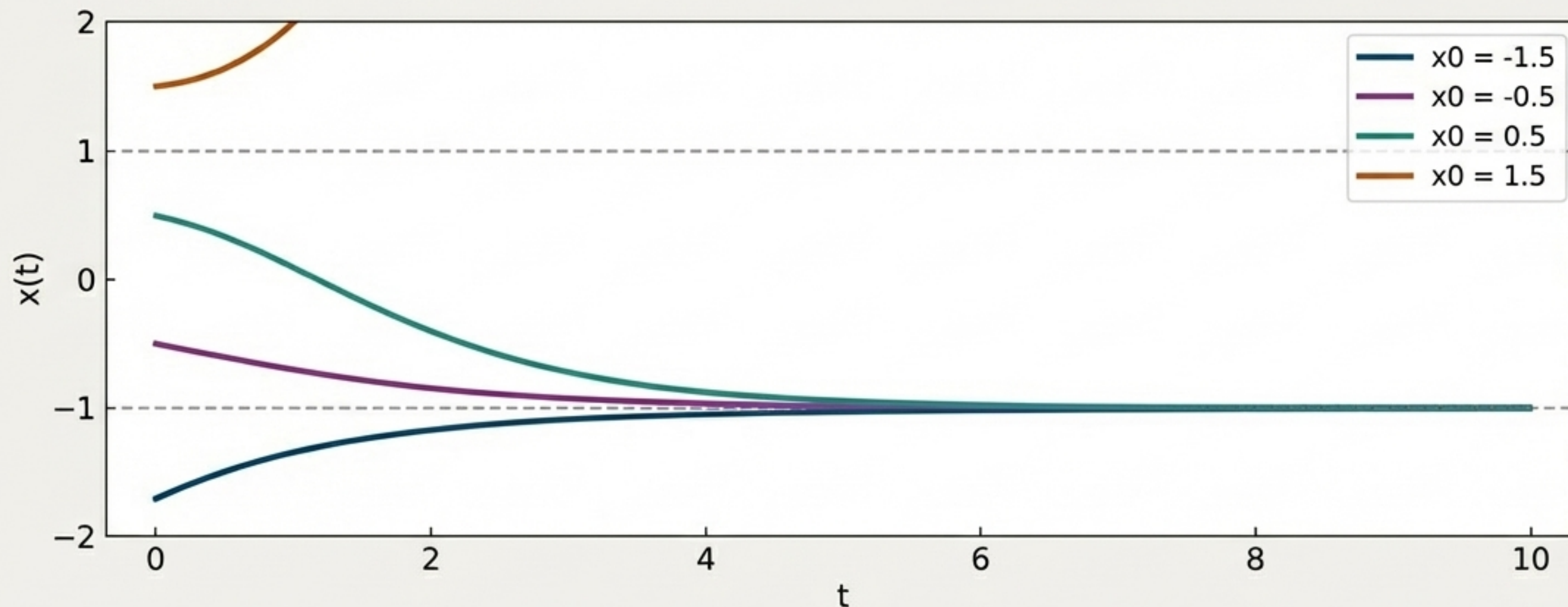
Este gráfico muestra $f(x)$ frente a x . Nos permite ver el "flujo" o la dirección del movimiento en toda la línea de estado.



Visualización 2: La Evolución en el Tiempo

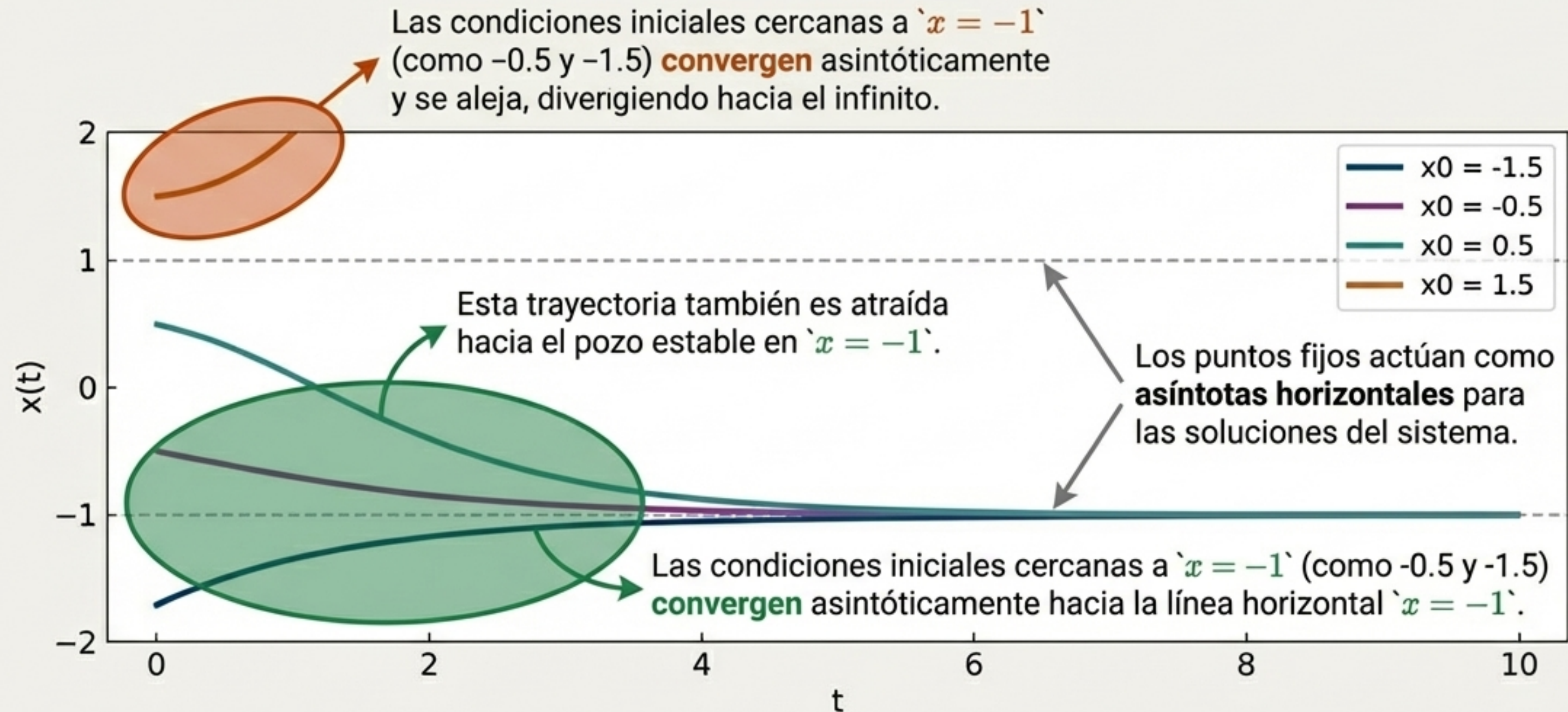
Aquí resolvemos numéricamente la ecuación diferencial $dx/dt = x^2 - 1$ para varias condiciones iniciales y graficamos $x(t)$ frente a t . Cada línea representa la "historia" de una partícula que comienza en un punto x_0 .

Condiciones Iniciales Simuladas: $x_0 = [-1.5, -0.5, 0.5, 1.5]$



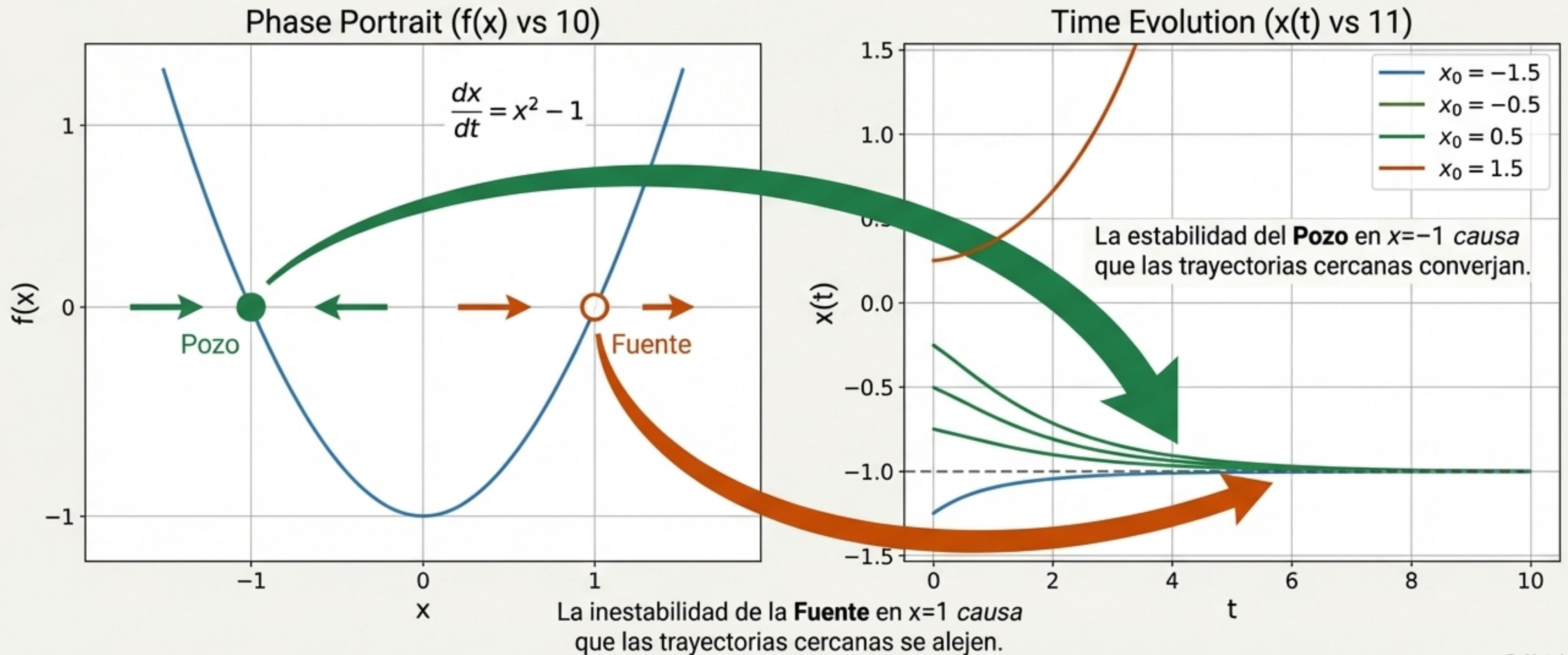
Deconstruyendo las Trayectorias

El comportamiento de las soluciones confirma nuestro análisis de estabilidad.



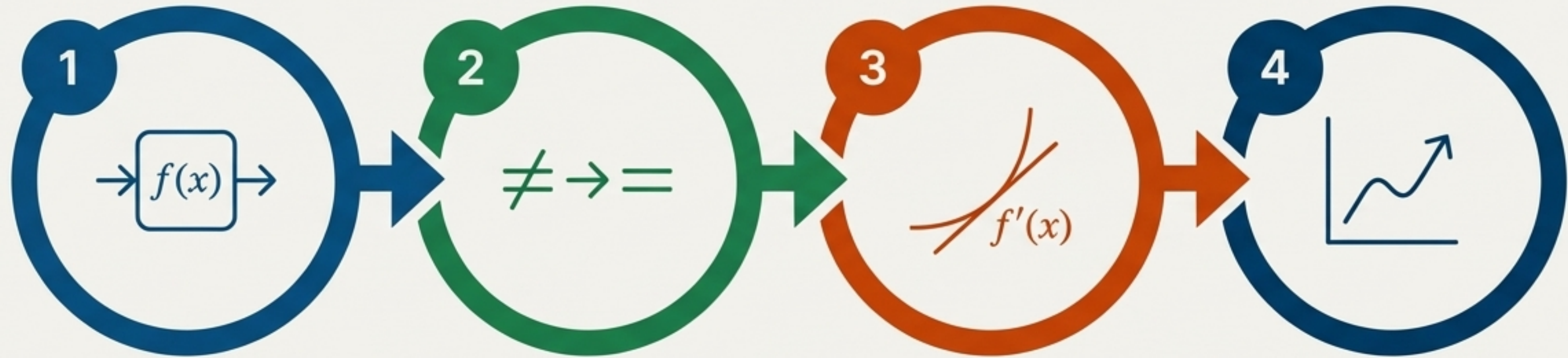
La Conexión Final: Causa y Efecto

Los dos gráficos cuentan la misma historia desde perspectivas diferentes. El retrato de fase explica el *porqué* del comportamiento que vemos en las trayectorias temporales.



Nuestro Flujo de Trabajo: Un Proceso Sistemático

Hemos seguido un proceso robusto y repetible para analizar el sistema autónomo:



Definir el Sistema:
Escribir la función $f(x)$
que gobierna la dinámica.

Encontrar Equilibrios:
Resolver $f(x) = 0$ para
hallar los puntos fijos.

Analizar Estabilidad:
Usar la derivada $f'(x)$
para clasificar cada
punto fijo.

Simular y Visualizar:
Resolver la EDO
numéricamente y crear
gráficos que revelen el
comportamiento.

Del Código a la Comprensión Visual

Con un script conciso de Python, hemos transformado una ecuación abstracta en una comprensión visual e intuitiva de la dinámica completa de un sistema. El código no es solo una herramienta de cálculo, sino un medio para explorar y entender.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve
from scipy.integrate import solve_ivp

# Define la función f(x)
def f(x):
    return x**2 - 1

# Define la derivada f'(x) para análisis de estabilidad
def f_prime(x):
    return 2 * x

def plot_function_with_fixes_and_solutions(f):
    # --- Parte 1: Gráfico de f(x) y Puntos Fijos ---
    x_plot = np.linspace(-2, 2, 400)
    y_plot = f(x_plot)

    # Encontrar puntos fijos numéricamente
    initial_guesses = [-1.5, 1.5]
    fixed_points = fsolve(f, initial_guesses)

    # Clasificar estabilidad
    stability = ['Estable' if f_prime(fp) < 0 else 'Inestable' for fp in fixed_points]

    # Crear la figura y los subgráficos
    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

    # Gráfico del retrato de fase
    ax1.plot(x_plot, y_plot, label=f'f(x) = x^2 - 1', color='008366', linewidth=2)
    ax1.axhline(0, color='000000', linestyle='--')
    ax1.set_title('Retrato de Fase (f(x) vs x)', fontsize=12, fontname='Lora', fontweight='bold')
    ax1.set_xlabel('x', fontsize=18, fontname='Inter')
    ax1.set_ylabel('f(x)', fontsize=18, fontname='Inter')

    # Marcar puntos fijos con colores de estabilidad
    for fp, stab in zip(fixed_points, stability):
        color = '#2E8448' if stab == 'Estable' else '#D9534F'
        marker = 'o' if stab == 'Estable' else 'o'
        label = 'Pozo' if stab == 'Estable' else 'Fuente'
        ax1.plot(fp, f(fp), marker, color, markersize=10, label=label, markeredgecolor='black', markeredgewidth=1)

    # Añadir flechas de flujo
    ax1.arrow(-1.5, 0, 0.3, 0, head_width=0.1, head_length=0.1, fc='008366', ec='008366')
    ax1.arrow(-0.5, 0, -0.3, 0, head_width=0.1, head_length=0.1, fc='008366', ec='008366')
    ax1.arrow(0.5, 0, 0.3, 0, head_width=0.1, head_length=0.1, fc='D9534F', ec='D9534F')
    ax1.arrow(1.5, 0, 0.3, 0, head_width=0.1, head_length=0.1, fc='D9534F', ec='D9534F')

    ax1.text(-1.3, -0.3, 'Pozo', fontsize=10, fontname='Inter', color='008366')
    ax1.text(1.3, -0.3, 'Fuente', fontsize=10, fontname='Inter', color='D9534F')
    ax1.legend()
    ax1.grid(True, color='000000', linestyle='--', linewidth=0.5)

    # Evolución temporal
    ax2.set_title('Evolución Temporal (x(t) vs t)', fontsize=12, fontname='Lora', fontweight='bold')
    ax2.set_xlabel('t', fontsize=18, fontname='Inter')
    ax2.set_ylabel('x(t)', fontsize=18, fontname='Inter')

    # Soluciones para x0 = -0.5
    x0 = -0.5
    t_span = (0, 10)
    t_eval = np.linspace(0, 10, 100)
    sol = solve_ivp(f, t_span, x0, t_eval=t_eval)

    ax2.plot(sol.t, sol.y[0], color='008366', linewidth=2, label=f'x0 = {x0}')

    # Líneas de equilibrio
    ax2.axhline(1, color='008366', linestyle='--', label='Equilibrio Estable (x=-1)')
    ax2.axhline(-1, color='D9534F', linestyle='--', label='Equilibrio Inestable (x=1)')

    plt.tight_layout()
    plt.show()

# Ejecutar las funciones
plot_function_with_fixes_and_solutions(f)
```

